

Unit-VI

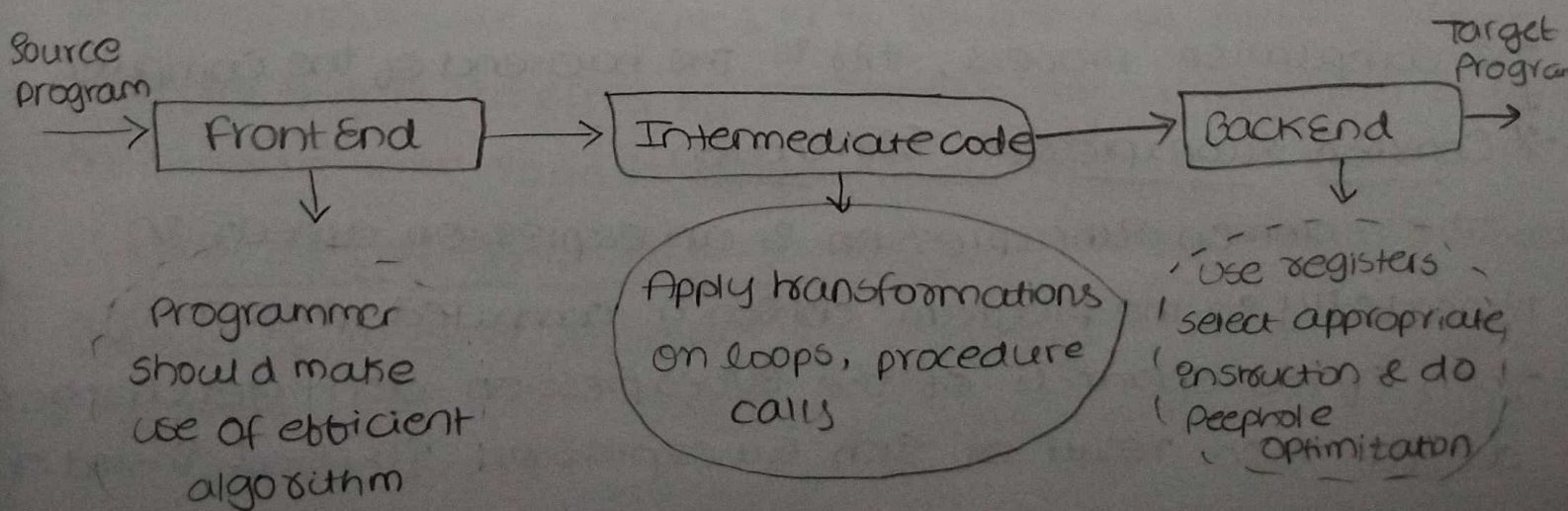
Run Time Environment: Storage organization, Runtime storage Allocation, Activation of Records, Procedure calls, Displays,

Code Optimization: The principle sources of optimization, Basic blocks, Optimization of Basic blocks, Structure preserving Transformations, Flow Graphs, Loop Optimization, Dataflow Analysis, peephole optimization.

Code Generation: Issues in the design of a code Generator, object code forms, Code Generation Algorithm, Register Allocation and Assignment.

Code Optimization: It required to produce an efficient target code. There are 2 important issues that need to be considered while applying the techniques for code optimisation.

1. The semantic equivalence of the source program must not be changed.
2. The improvement over the program efficiency must be achieved without changing the algorithm of the program.



* Code Optimization

Classification of optimization

It can be done in two categories:- 1) machine dependent optimization

2) machine independent optimization.

Machine independent optimization:-

Independent of program transformations, that improve target code without taking any considerations of properties of target machine.

Use appropriate program structure in order to improve the efficiency of target code. From the source program eliminate unreachable code.

Machine dependent optimization:-

This optimization is based on register allocation and utilization of special machine instruction sequence. Allocation of subroutines, no. of resources to improve the execution efficiency of the program.

Using immediate instructions wherever necessary.

* Principal sources of optimization (or) Techniques

This is one of the code optimization, it is one of the level of the compilation process, this is the backend of the compiler.

* Common Subexpression Elimination:-

The common subexpression is an expression appear repeatedly in the program which is computed previously.

Then if the operands of the subexpression don't get changed at all then result of such subexpression is used instead of re computing it each time.

Ex:	Unoptimised code	Optimised code
	$a = b + c$	$a = b + c$
	$b = a - d$	$b = a - d$
	$c = b + c$	$c = b + c$
	$d = a - d$	$b \neq d = b$

$$t_1 = 4 * i$$

$$t_2 = a[t_1]$$

$$t_3 = 4 * j$$

$$t_4 = 4 * i$$

$$t_5 = n_1$$

$$t_6 = b[t_4] + t_5$$

The code can be Optimised with the Common Subexpression elimination as

The common sub-expression

$t_4 = 4 * i$ is eliminated

as its computation is

already in t_1 & value of 'i'

not been changed from definition to use.

$$t_1 = 4 * i$$

$$t_2 = a[t_1]$$

$$t_3 = 4 * j$$

$$t_5 = n_1$$

$$t_6 = b[t_1] + t_5$$

* Compile-Time Evaluation:

Evaluation is done at Compile time instead of runtime.

a) Constant Folding: In the folding technique, the computation of constant is done at compile time instead of execution time.

Ex: $\text{area} = (22/7) * r * r$

$22/7$ is calculated & result 3.14 is replaced. so $\text{area} = 3.14 * r * r$.

Here folding is implied by performing the computation of $22/7$ at compile time instead of execution time.

b) Constant (or) copy propagation: It means use of one variable instead of another variable.

Ex: $x = \pi$

$$\text{area} = x * r * r$$

The optimisation using copy propagation can be done as follows:

$$\text{area} = \pi * r * r$$

Here the variable 'x' is eliminated. Hence the necessary condition

is that a variable must be assigned to another variable (or)

some constant

3.

* Code Movement:

It is a technique which moves the code outside the loop if it own have any difference, & execute inside (or) outside the loop.

Ex: for(i=0; i<n; i++)

{

 x = y + z; #before

 a[i] = 6 * i;

}

x = y + z # after

for(i=0; i<len; i++)

{

 a[i] = 6 * i;

}

⇒

4.

* Dead Code elimination

It includes eliminate those statements which are never executed (or) & execute output is never used.

Ex: unoptimised

i = 0;

if (i == 1)

{

 a++;

}

optimised

i = 0;

5.

* Strength Reduction

The strength of certain operators is higher than the others. For instance, strength of '*' is higher than '+'. In strength reduction technique, the higher strength operator can be replaced by lower strength operator.

Ex: Unoptimised

for(i=1; i<=50; i++)

{

 count = i * 2;

}

Optimised

temp = 7;

for(i=0; i<=50; i++)

{

 count = temp;

}

temp = temp + 2;

NOTE: It is used only when higher strength operator & lower strength operator gives the same output.

* Basic blocks & Flowgraphs:

Algorithm for partitioning Three-address code into basic blocks

- Rule 1: Determining the leaders, 1) First statement is a leader
2) The target of conditional or unconditional jump is a leader.
3) The statement following condition (or) unconditional jump is leader

Ex: 1) $PROD = 0$

2) $T = 1$

3) $T_2 = \text{addr}(A) - 4$

4) $T_4 = \text{addr}(B) - 4$

5) $T_1 = 4 * i$

6) $T_3 = T_2[T_1]$

7) $Prod = prod + T_3$

8) $I = I + 1$

9) IF $I \leq 20$ goto(5)

10) $j = j + 1$

11) $K = K + 1$

12) IF $j \leq 5$ goto(7)

13) $i = i + 1$

